

本节内容

带符号整数的 乘法运算原理

知识总览

带符号整数的
乘法运算原理

带符号数的乘法电路结构、运算过程

带符号整数乘法的溢出判断

Intro: C语言中的乘法示例

【例】无符号整数乘法

```
unsigned int x=985;  
unsigned int y=211;  
unsigned int z = x * y;
```

C语言 unsigned int型，
32 bit 无符号整数

【例】带符号整数乘法

```
int x= -985;  
int y= 211;  
int z = x * y;
```

C语言 int 型，32bit 带符号
整数，用补码表示

无符号整数乘法 vs 带符号整数乘法

```
  1101
× 1011
-----
 1101
 1101
 0000
 1101
-----
10001111
```

用电路模拟手算过程

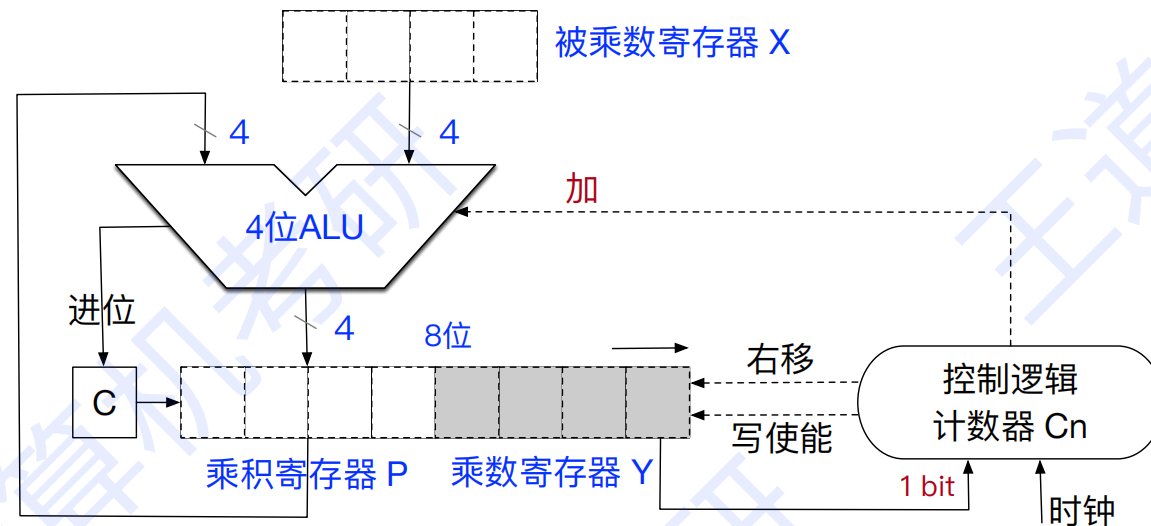
用十进制验算：
 $13 * 11 = 143$

按照无符号整数解读二进制串：

被乘数 $X = 1101 = 13$ (十进制)
乘数 $Y = 1011 = 11$ (十进制)
乘积 $Z = 10001111 = 143$ (十进制)



靓仔の肯定



无符号整数乘法电路

按照带符号整数 (补码) 解读二进制串：

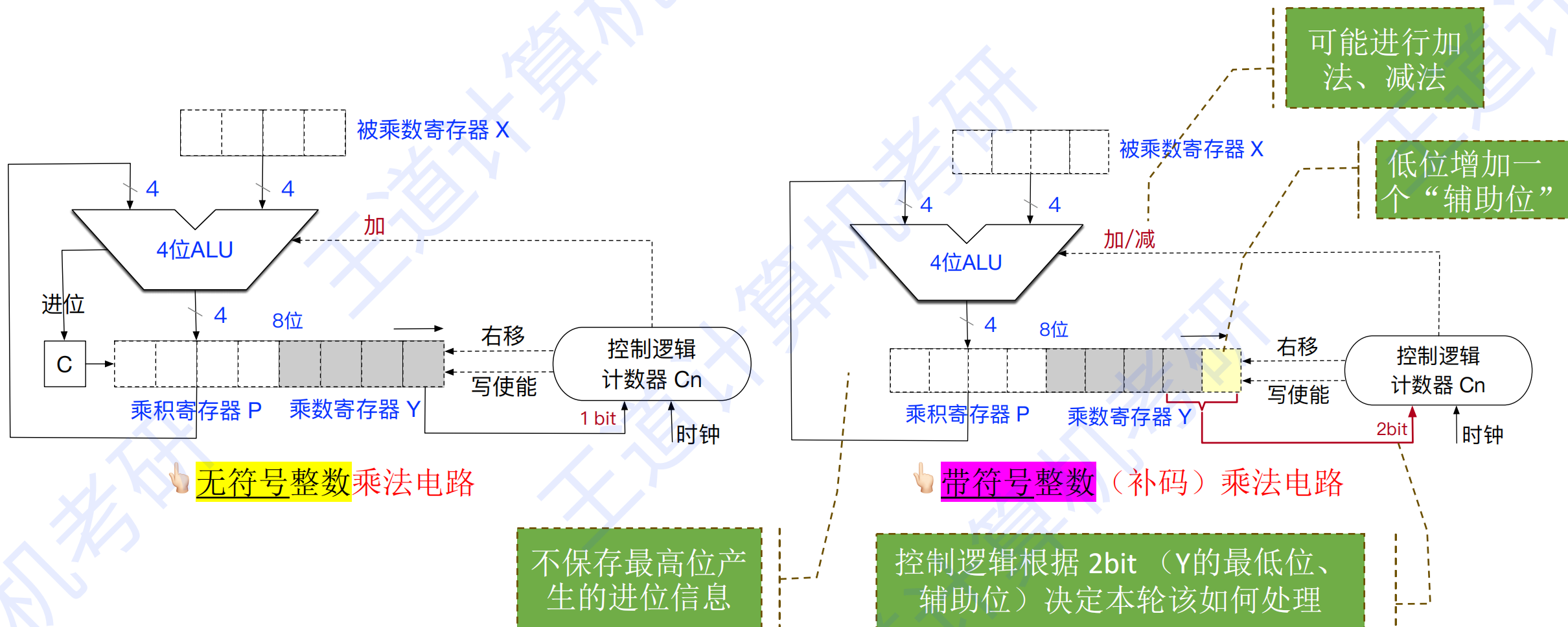
被乘数 $X = 1101$ (补码) = -3 (十进制)
乘数 $Y = 1011$ (补码) = -5 (十进制)
乘积 $Z = 10001111$ (补码) = -113 (十进制)

使用无符号整数乘法电路处理带符号整数 (补码) 乘法，结果明显错误



错!

无符号整数 vs 带符号整数乘法电路

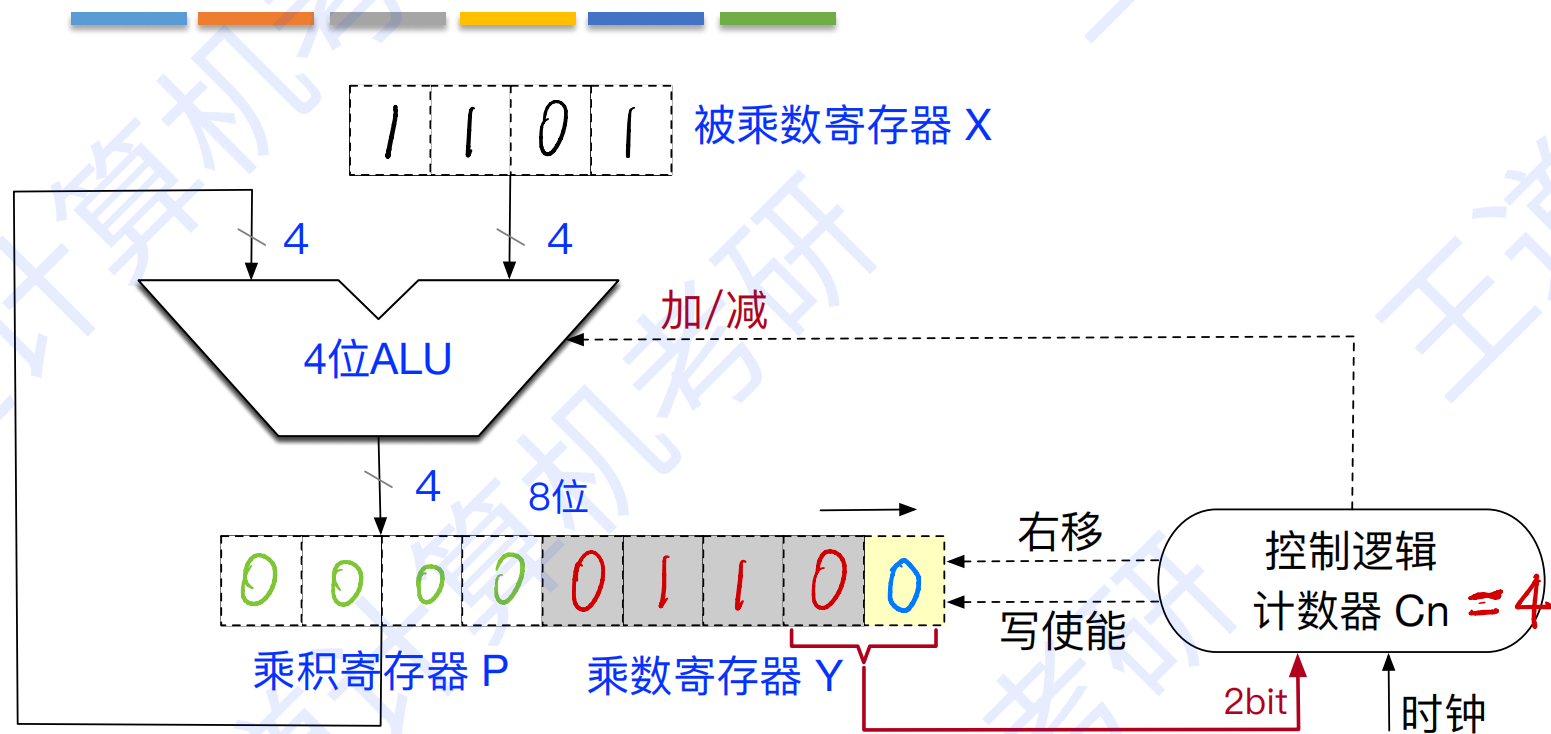


带符号整数乘法原理：以 4bit 带符号整数为例

$$\begin{array}{r} 1101 \rightarrow X_{\text{补}} \rightarrow -3 \\ \times 0110 \rightarrow Y_{\text{补}} \rightarrow 6 \\ \hline 11101110 \rightarrow Z_{\text{补}} \rightarrow -18 \end{array}$$

注1：由于带符号整数使用补码表示，因此二进制运算的数学原理要比无符号整数更复杂。为了降低大家的备考负担，我们将聚焦于考研重点，仅介绍运算过程，不探讨数学原理。

注2：带符号数（补码）乘法，符号位参与运算



开始：

1. 将被乘数、乘数 分别放入寄存器X、Y
2. 乘积寄存器P 置为0，“辅助位”置为0
3. 计数器 C_n 的初始值置为 n （n 为乘数的位数）

特殊情况：当被乘数、乘数中有一个为全0时，结果直接得0，不需要再进行后续的运算步骤

带符号整数乘法原理：以 4bit 带符号整数为例

$$\begin{array}{r} 1101 \rightarrow X_{\text{补}} \rightarrow -3 \\ \times 0110 \rightarrow Y_{\text{补}} \rightarrow 6 \\ \hline 11101110 \rightarrow Z_{\text{补}} \rightarrow -18 \end{array}$$

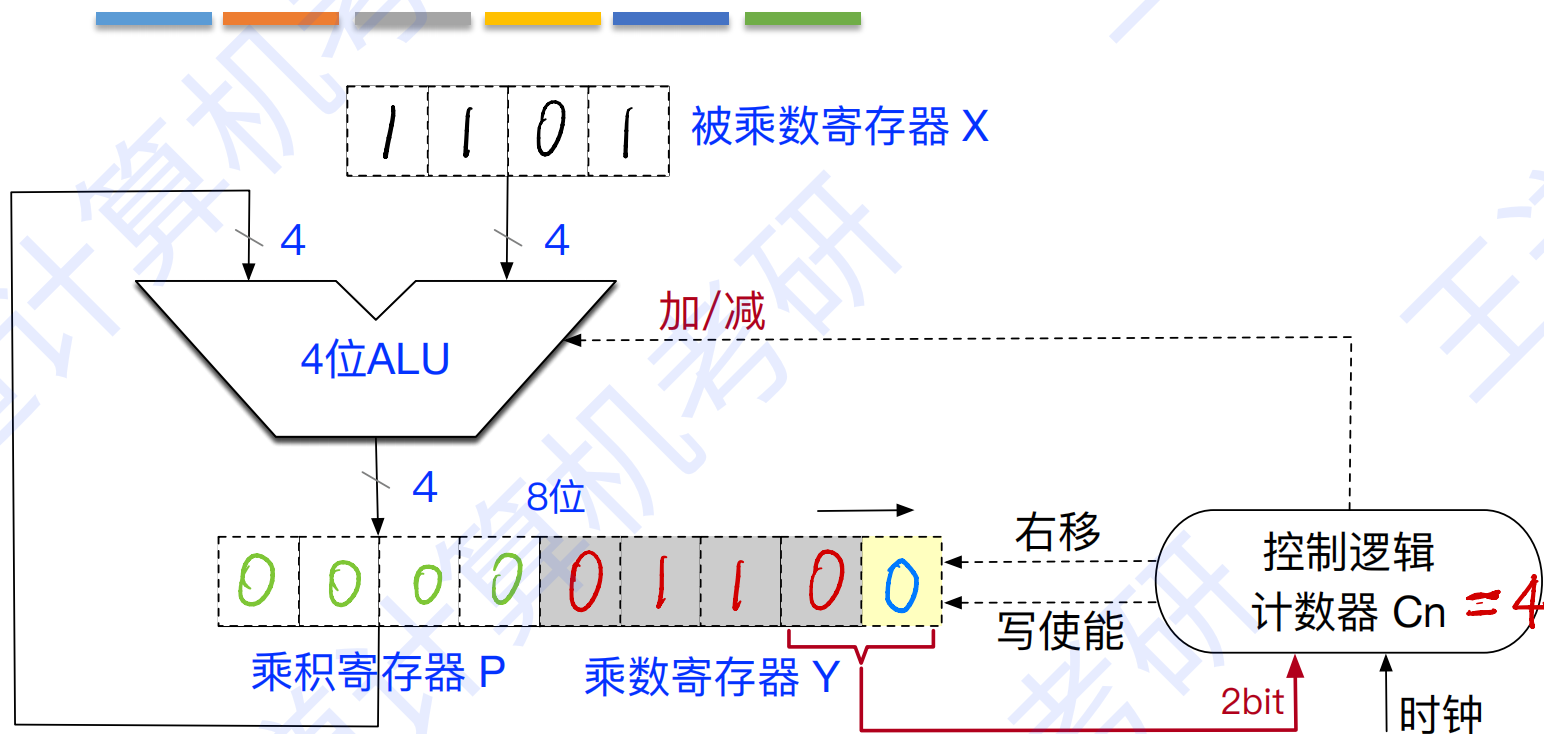
寄存器Y 最低位	辅助位	本轮操作
0	0	+0
0	1	+ [x] _补
1	0	- [x] _补
1	1	+0

记忆技巧：

辅助位 - Y最低位 = 1, $P + [x]_{\text{补}}$

辅助位 - Y最低位 = 0, $P + 0$

辅助位 - Y最低位 = -1, $P - [x]_{\text{补}}$



重复 n 轮 **加/减法**、**移位** 运算，直到计数器 $C_n = 0$ ：

1. 将**乘数寄存器Y的最低位**、**辅助位**，**2bit** 送入“**控制逻辑**”进行判断
2. 根据**寄存器Y的最低位**、**辅助位**，决定是“ **$+ [x]_{\text{补}}$** ” “ **$- [x]_{\text{补}}$** ” “ **$+0$** ”
3. 将【P, Y, 辅助位】视为整体，**算数右移一位**
4. 计数器 C_n 减1

带符号整数乘法原理：以 4bit 带符号整数为例

$$\begin{array}{r}
 1101 \rightarrow X_{\text{补}} \rightarrow -3 \\
 \times 0110 \rightarrow Y_{\text{补}} \rightarrow 6 \\
 \hline
 11101110 \rightarrow Z_{\text{补}} \rightarrow -18
 \end{array}$$

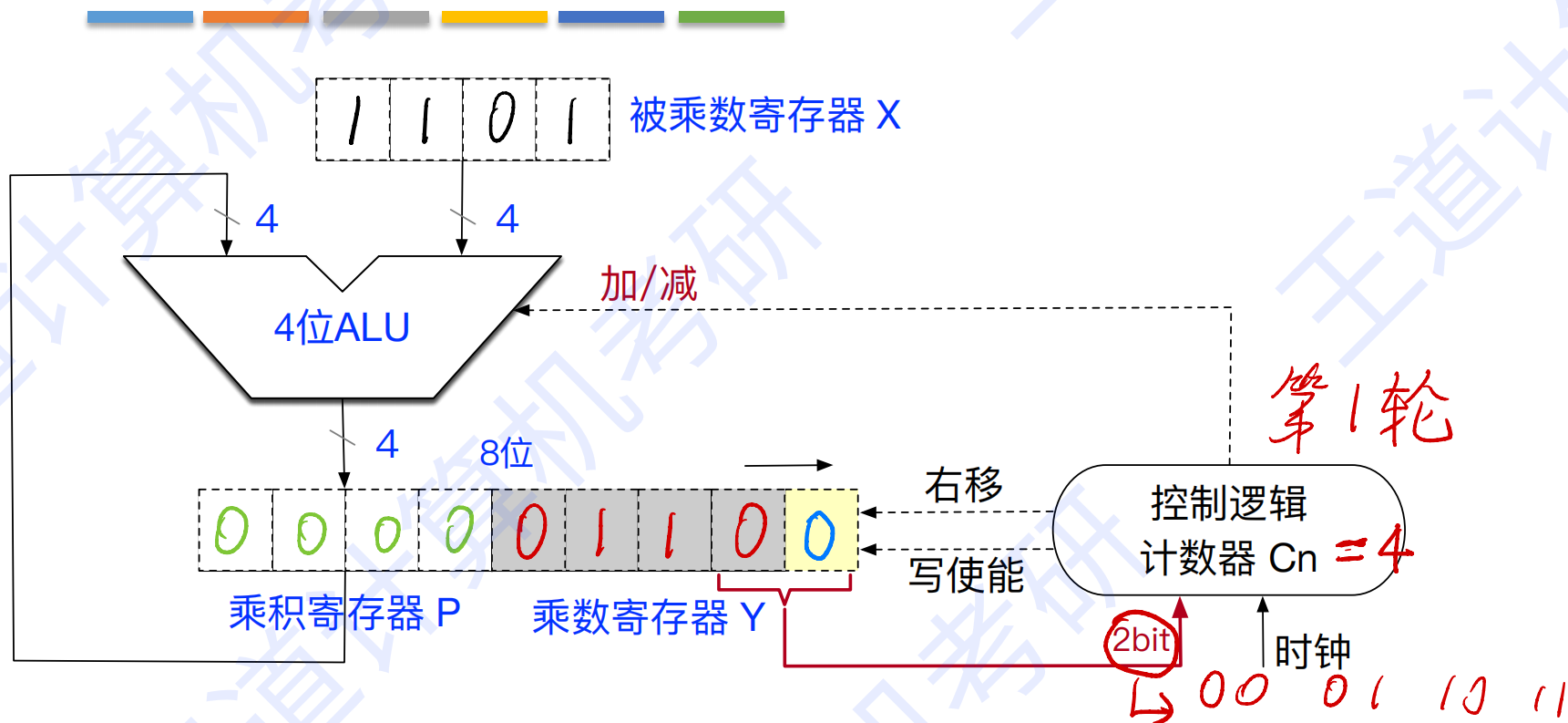
寄存器Y 最低位	辅助位	本轮操作
0	0	+0
0	1	+ $[x]_{\text{补}}$
1	0	- $[x]_{\text{补}}$
1	1	+0

记忆技巧：

辅助位 - Y最低位 = 1, $P + [x]_{\text{补}}$

辅助位 - Y最低位 = 0, $P + 0$

辅助位 - Y最低位 = -1, $P - [x]_{\text{补}}$



重复 n 轮 加/减法、移位运算，直到计数器 $C_n = 0$ ：

1. 将乘数寄存器Y的最低位、辅助位，2bit 送入“控制逻辑”进行判断
2. 根据寄存器Y的最低位、辅助位，决定是“+ $[x]_{\text{补}}$ ” “- $[x]_{\text{补}}$ ” “+0”
3. 将【P, Y, 辅助位】视为整体，算数右移一位
4. 计数器 C_n 减1

带符号整数乘法原理：以 4bit 带符号整数为例

$$\begin{array}{r}
 1101 \rightarrow X_{\text{补}} \rightarrow -3 \\
 \times 0110 \rightarrow Y_{\text{补}} \rightarrow 6 \\
 \hline
 11101110 \rightarrow Z_{\text{补}} \rightarrow -18
 \end{array}$$

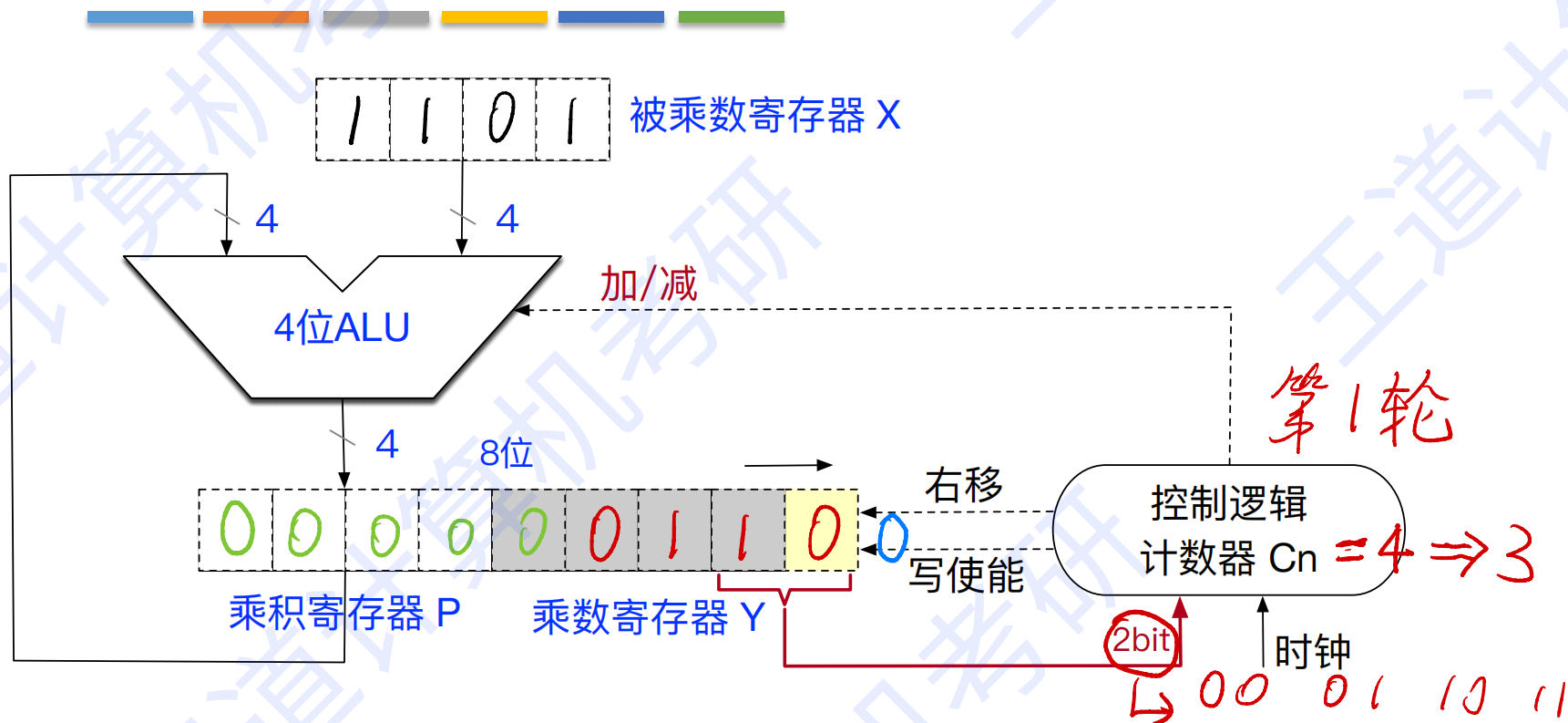
寄存器Y 最低位	辅助位	本轮操作
0	0	+0
0	1	+ $[x]_{\text{补}}$
1	0	- $[x]_{\text{补}}$
1	1	+0

记忆技巧：

辅助位 - Y最低位 = 1, $P + [x]_{\text{补}}$

辅助位 - Y最低位 = 0, $P + 0$

辅助位 - Y最低位 = -1, $P - [x]_{\text{补}}$



重复 n 轮 加/减法、移位运算，直到计数器 $C_n = 0$ ：

1. 将乘数寄存器Y的最低位、辅助位，2bit 送入“控制逻辑”进行判断 ✓
2. 根据寄存器Y的最低位、辅助位，决定是“+ $[x]_{\text{补}}$ ” “- $[x]_{\text{补}}$ ” “+0” ✓
3. 将【P, Y, 辅助位】视为整体，算数右移一位 ✓
4. 计数器 C_n 减1 ✓

带符号整数乘法原理：以 4bit 带符号整数为例

$$\begin{array}{r} 1101 \rightarrow X_{\text{补}} \rightarrow -3 \\ \times 0110 \rightarrow Y_{\text{补}} \rightarrow 6 \\ \hline 11101110 \rightarrow Z_{\text{补}} \rightarrow -18 \end{array}$$

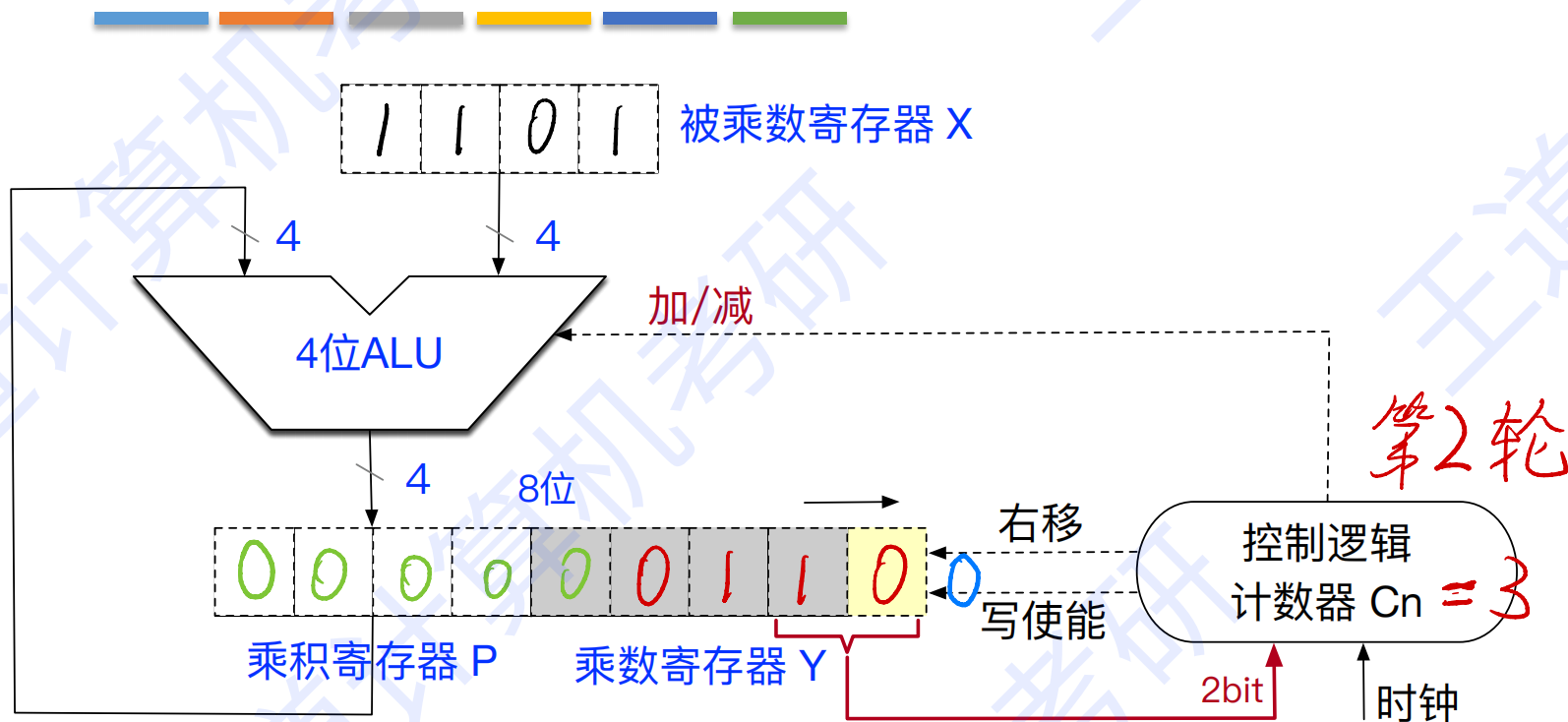
寄存器Y 最低位	辅助位	本轮操作
0	0	+0
0	1	+ $[x]_{\text{补}}$
1	0	- $[x]_{\text{补}}$
1	1	+0

记忆技巧：

辅助位 - Y最低位 = 1, $P + [x]_{\text{补}}$

辅助位 - Y最低位 = 0, $P + 0$

辅助位 - Y最低位 = -1, $P - [x]_{\text{补}}$



重复 n 轮 加/减法、移位运算，直到计数器 $C_n = 0$ ：

1. 将乘数寄存器Y的最低位、辅助位，2bit 送入“控制逻辑”进行判断
2. 根据寄存器Y的最低位、辅助位，决定是“+ $[x]_{\text{补}}$ ”“- $[x]_{\text{补}}$ ”“+0”
3. 将【P, Y, 辅助位】视为整体，算数右移一位
4. 计数器 C_n 减1

带符号整数乘法原理：以 4bit 带符号整数为例

$1101 \rightarrow X_{补} \rightarrow -3$
 $\times 0110 \rightarrow Y_{补} \rightarrow 6$

 $11101110 \rightarrow Z_{补} \rightarrow -18$

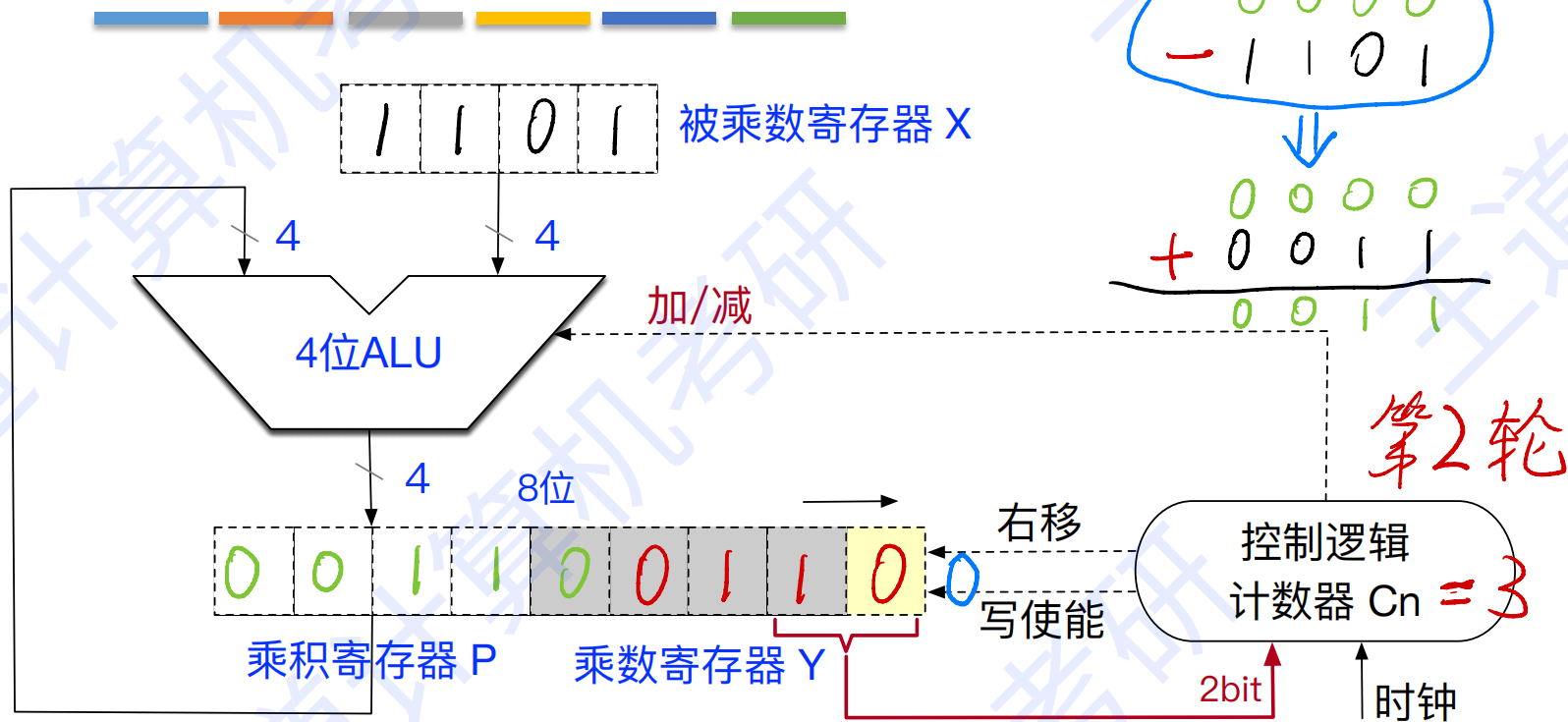
寄存器Y 最低位	辅助位	本轮操作
0	0	+0
0	1	+ $[x]_{补}$
1	0	- $[x]_{补}$
1	1	+0

记忆技巧：

辅助位 - Y最低位 = 1, $P + [x]_{补}$

辅助位 - Y最低位 = 0, $P + 0$

辅助位 - Y最低位 = -1, $P - [x]_{补}$



重复 n 轮 加/减法、移位运算，直到计数器 $C_n = 0$ ：

1. 将乘数寄存器Y的最低位、辅助位，2bit 送入“控制逻辑”进行判断 ✓
2. 根据寄存器Y的最低位、辅助位，决定是“+ $[x]_{补}$ ”“- $[x]_{补}$ ”“+0” ✓
3. 将【P, Y, 辅助位】视为整体，算数右移一位
4. 计数器 C_n 减1

带符号整数乘法原理：以 4bit 带符号整数为例

$1101 \rightarrow X_{补} \rightarrow -3$
 $\times 0110 \rightarrow Y_{补} \rightarrow 6$

 $11101110 \rightarrow Z_{补} \rightarrow -18$

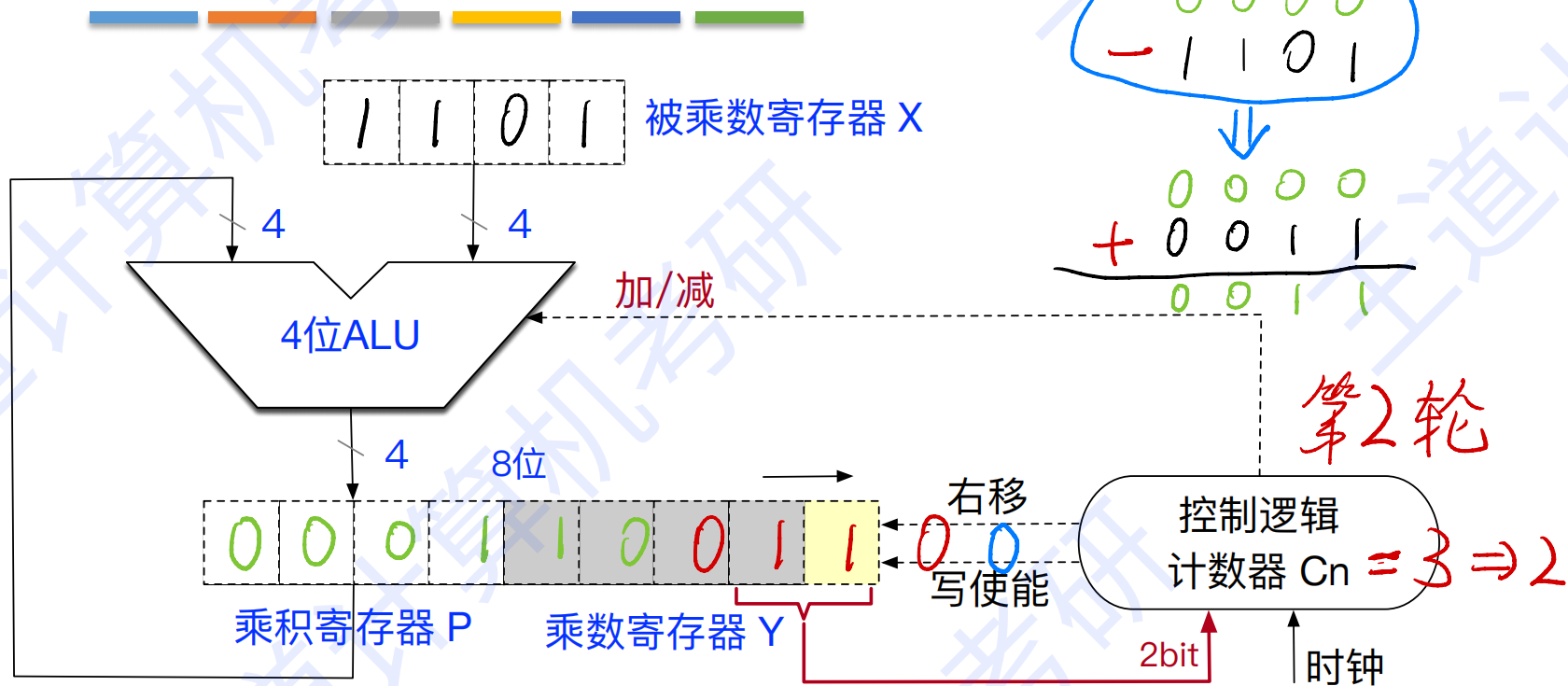
寄存器Y 最低位	辅助位	本轮操作
0	0	+0
0	1	+ $[x]_{补}$
1	0	- $[x]_{补}$
1	1	+0

记忆技巧：

辅助位 - Y最低位 = 1, $P + [x]_{补}$

辅助位 - Y最低位 = 0, $P + 0$

辅助位 - Y最低位 = -1, $P - [x]_{补}$



重复 n 轮 加/减法、移位运算，直到计数器 $C_n = 0$ ：

1. 将乘数寄存器Y的最低位、辅助位，2bit 送入“控制逻辑”进行判断 ✓
2. 根据寄存器Y的最低位、辅助位，决定是“+ $[x]_{补}$ ” “- $[x]_{补}$ ” “+0” ✓
3. 将【P, Y, 辅助位】视为整体，算数右移一位 ✓
4. 计数器 C_n 减1 ✓

$\times 0110 \rightarrow Y_{\text{补}} \rightarrow 6$



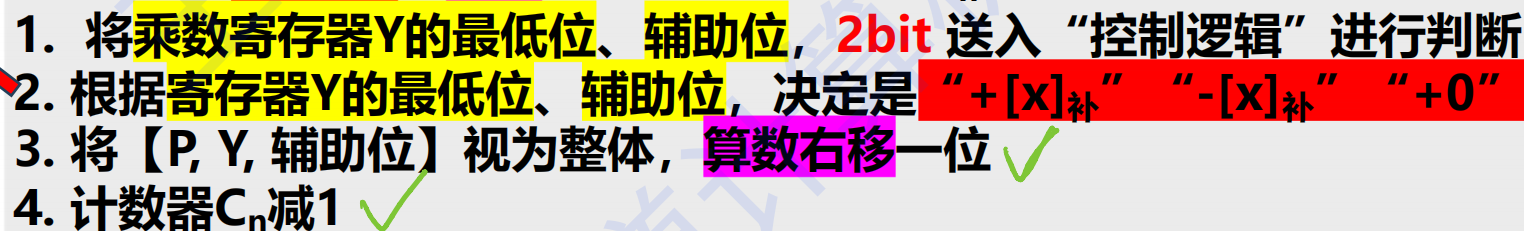
0	0	+0
0	1	+ [x] _补
1	0	- [x] _补
1	1	+0

辅助位 - Y最低位 = 1, $P + [x]_{\text{补}}$
 辅助位 - Y最低位 = 0, $P + 0$
 辅助位 - Y最低位 = -1, $P - [x]_{\text{补}}$

1. 将乘数寄存器Y的最低位、辅助位，2bit 送入“控制逻辑”进行判断
2. 根据寄存器Y的最低位、辅助位，决定是“ $+ [x]_{\text{补}}$ ” “ $- [x]_{\text{补}}$ ” “ $+ 0$ ”
3. 将【P, Y, 辅助位】视为整体，算数右移一位
4. 计数器 C_n 减1

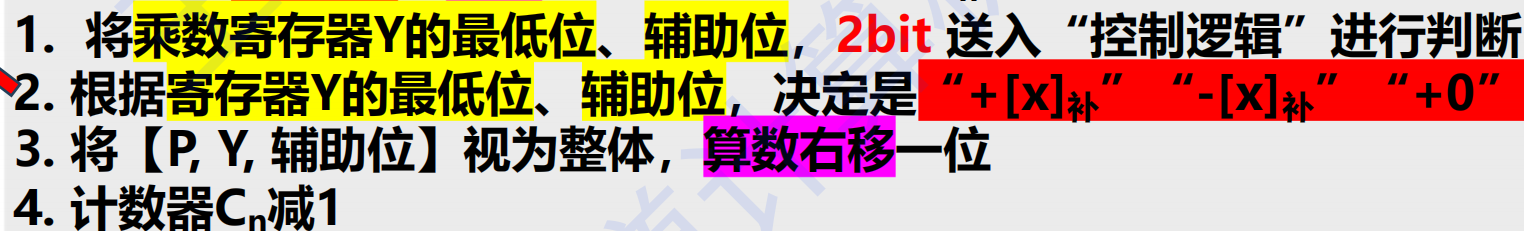
$11101110 \rightarrow Z_{\text{补}} \rightarrow -18$

辅助位 - Y最低位 = 1, $P + [x]_{\text{补}}$
 辅助位 - Y最低位 = 0, $P + 0$
 辅助位 - Y最低位 = -1, $P - [x]_{\text{补}}$



$11101110 \rightarrow Z_{\text{补}} \rightarrow -18$

辅助位 - Y最低位 = 1, $P + [x]_{\text{补}}$
 辅助位 - Y最低位 = 0, $P + 0$
 辅助位 - Y最低位 = -1, $P - [x]_{\text{补}}$



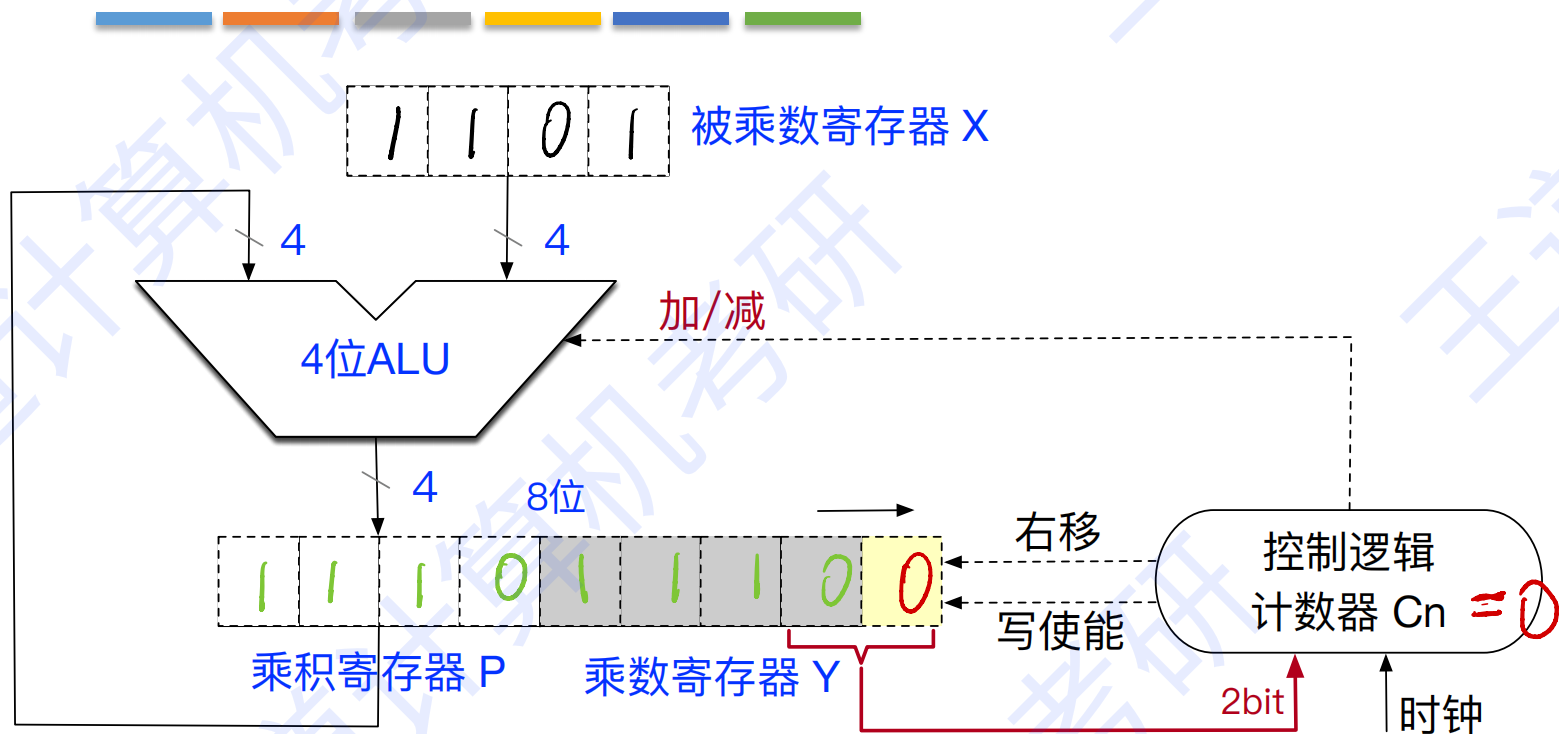
$11101110 \rightarrow Z_{\text{补}} \rightarrow -18$

辅助位 - Y最低位 = -1, P - [x]_补



带符号整数乘法原理：以 4bit 带符号整数为例

$$\begin{array}{r} 1101 \rightarrow X_{\text{补}} \rightarrow -3 \\ \times 0110 \rightarrow Y_{\text{补}} \rightarrow 6 \\ \hline 11101110 \rightarrow Z_{\text{补}} \rightarrow -18 \end{array}$$



结束 (计数器 $C_n = 0$) :

- ① 乘法运算的结果用 $2n$ 位暂存 (寄存器P、寄存器Y)
- ② 在很多计算机架构中, 通常仅保留低 n 位作为乘积结果 (因此, 运算结果可能发生溢出)

带符号整数乘法原理：溢出判断、溢出处理

例1: $1101 \times 0110 = 1110\ 1110$

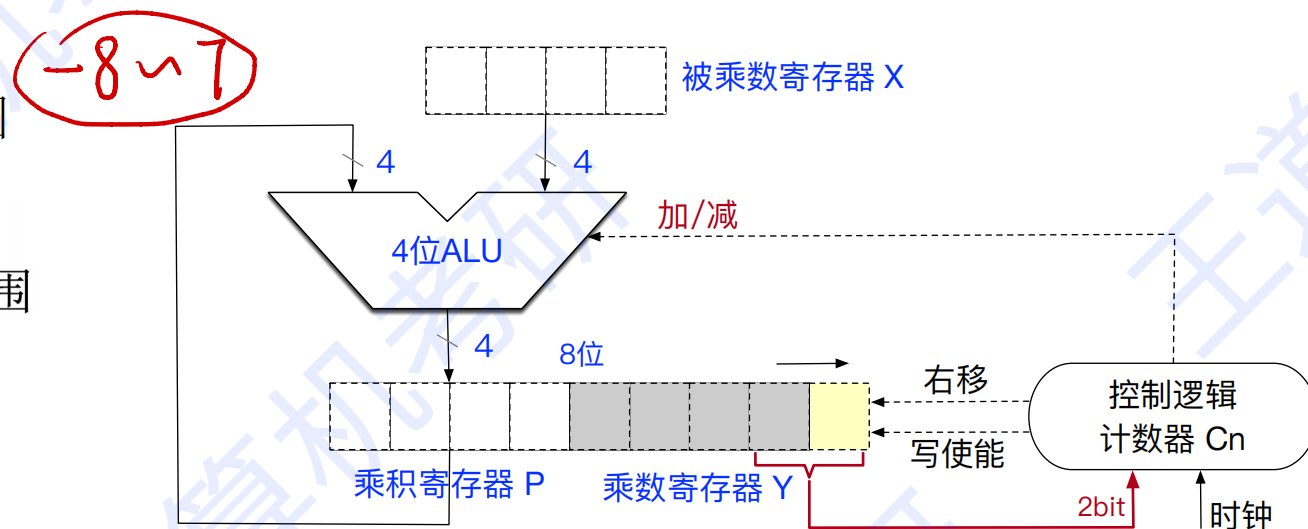
$-3 \times 6 = -18$, 超出4bit带符号数表示范围

例2: $1101 \times 0010 = 1111\ 1010$

$-3 \times 2 = -6$, 未超出4bit带符号数表示范围

例3: $0100 \times 0010 = 0000\ 1000$

$4 \times 2 = 8$, 超出4bit带符号数表示范围



带符号整数（补码）乘法的**溢出判断**:

- 两个 n bit **带符号整数**相乘，运算**结果用 $2n$ bit 暂存**。通常仅保留低 n 位作为乘法运算结果。若**高 $n+1$ 位不完全相同**，说明**发生溢出**，此时可将**OF标志位（溢出标志位）**置为1

溢出的处理:

- ✓ 程序员可以选择**忽略“溢出”**，只不过这样会导致错误的运算结果
- ✓ 如果想要**处理“溢出”**这种**“异常”**，可以在**乘法指令之后**执行一条**溢出自陷指令**，（例如：x86 的 INTO 指令（**Interrupt on Overflow**））。**该指令会检查 OF标志位，若 $OF==1$ ，就执行操作系统的“异常处理程序”**

真题训练：2010年_13

13. 假定有 4 个整数用 8 位补码分别表示 $r1 = \text{FEH}$, $r2 = \text{F2H}$, $r3 = 90\text{H}$, $r4 = \text{F8H}$ 。若将运算结果存放在一个 8 位寄存器中，则下列运算中会发生溢出的是（ ）。

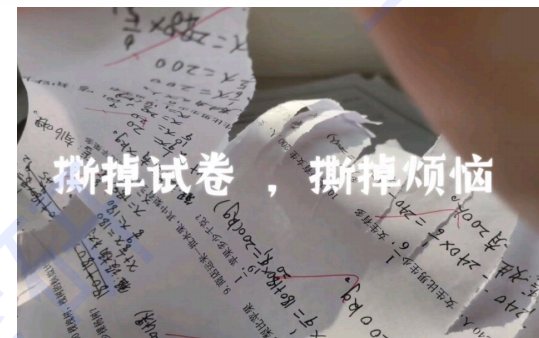
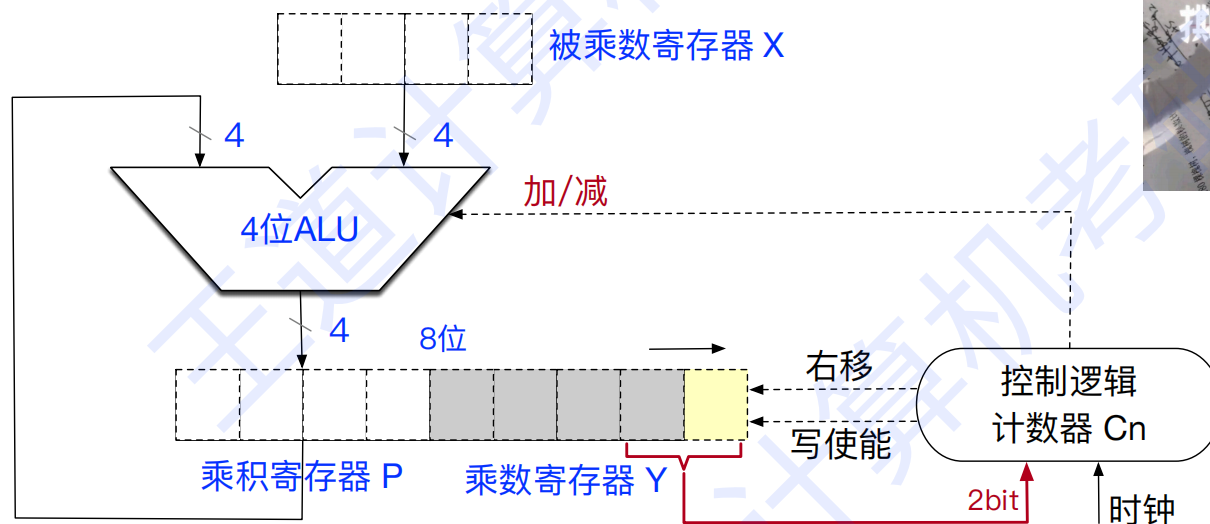
A. $r1 \times r2$

B. $r2 \times r3$

C. $r1 \times r4$

D. $r2 \times r4$

解题思路：画一个 $8\text{bit} \times 8\text{bit}$ 带符号整数乘法器，逐步模拟乘法过程。根据高 $n+1$ bit 是否相同来判断是否溢出！



我的内心几乎是崩溃的

真题训练：2010年_13

13. 假定有 4 个整数用 8 位补码分别表示 $r1 = \text{FEH}$, $r2 = \text{F2H}$, $r3 = 90\text{H}$, $r4 = \text{F8H}$ 。若将运算结果存放在一个 8 位寄存器中，则下列运算中会发生溢出的是 ()。

A. $r1 \times r2$

B. $r2 \times r3$

C. $r1 \times r4$

D. $r2 \times r4$

8bit 带符号整数（补码）范围 $-128 \sim 127$

$r1 = -2$

$r2 = -14$

$r3 = -112$

$r4 = -8$



$r2 \times r3 = 1568$



00000110 00100000

超出8bit补码合法范围，必定溢出

机器做法：16bit 暂存乘法运算结果，若高 n+1 bit 不完全相同，说明溢出

真题训练：2019年_45

45. (16 分) 已知 $f(n) = n! = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$, 计算 $f(n)$ 的 C 语言函数 f1 的源程序 (阴影部分) 及其在 32 位计算机 M 上的部分机器级代码如下:

```
int f1(int n){
1  00401000  55                push ebp
  ...
  if(n>1)
11 00401018  83 7D 08 01  cmp dword ptr [ebp+8],1
12 0040101C  7E 17        jle f1+35h (00401035)
    return n*f1(n-1);
13 0040101E  8B 45 08     mov eax, dword ptr [ebp+8]
14 00401021  83 E8 01     sub eax, 1
15 00401024  50          push eax
16 00401025  E8 D6 FF FF FF call f1 ( 00401000)
  ...
19 00401030  0F AF C1     imul eax, ecx
20 00401033  EB 05        jmp f1+3Ah (0040103a)
  else return 1;
21 00401035  B8 01 00 00 00 mov eax,1
  }
  ...
26 00401040  3B EC        cmp ebp, esp
  ...
30 0040104A  C3          ret
```

其中, 机器级代码行包括行号、虚拟地址、机器指令和汇编指令, 计算机 M 按字节编址, int 型数据占 32 位。请回答下列问题:

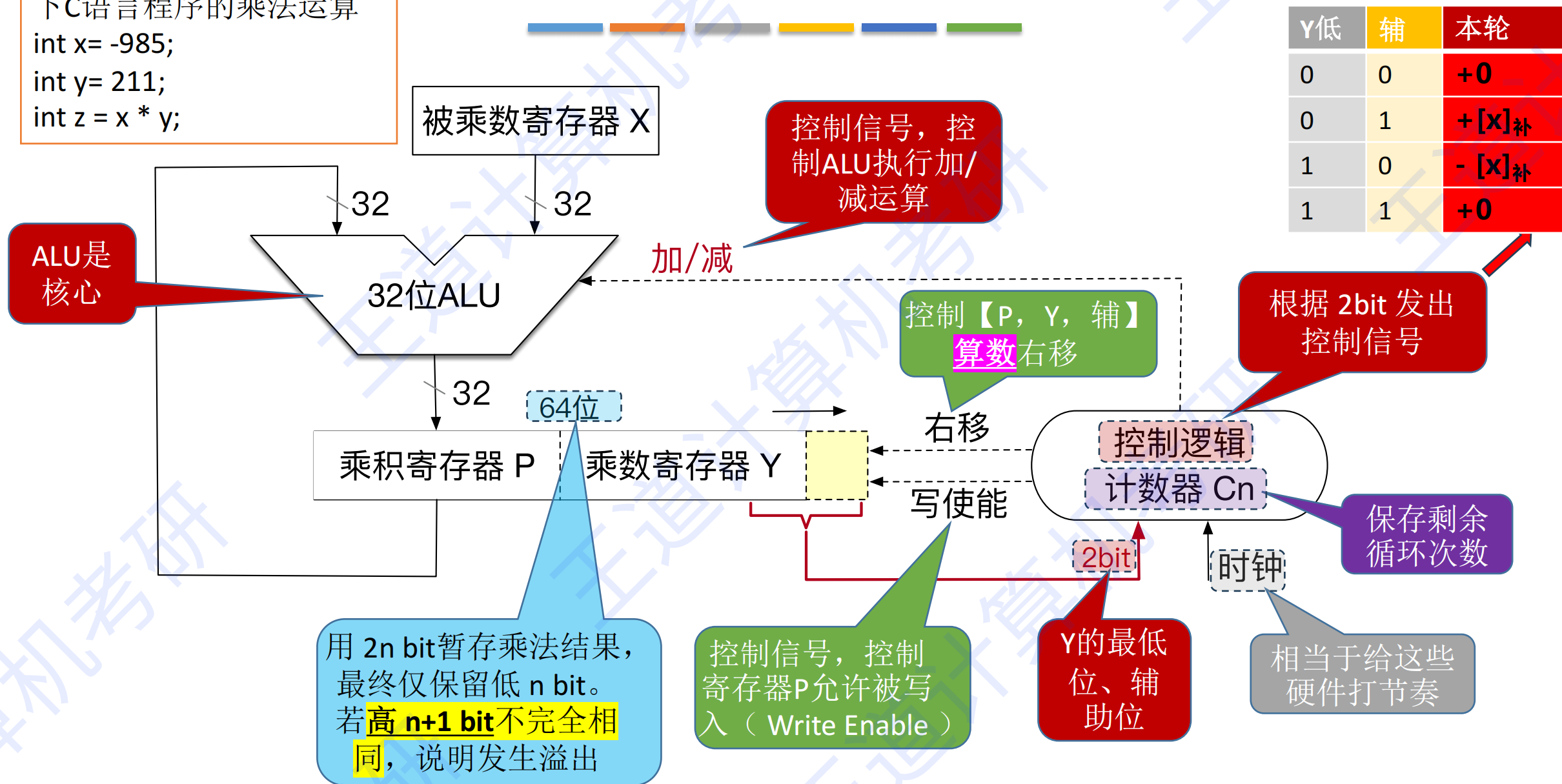
- 5) 第 19 行 imul 指令 (带符号整数乘) 的功能是 $R[ecx] \leftarrow R[ecx] \times R[ecx]$, 当乘法器输出的高、低 32 位乘积之间满足什么条件时, 溢出标志 OF = 1? 要使 CPU 在发生溢出时转异常处理, 编译器应在 imul 指令后应加一条什么指令?



例如：可用该电路实现如下C语言程序的乘法运算

```
int x= -985;  
int y= 211;  
int z = x * y;
```

32 位带符号数乘法运算电路的结构



知识回顾与重要考点

带符号整数的乘法运算原理

开始

①被乘数、乘数 分别放入寄存器X、Y；②乘积寄存器P 置为0，**辅助位置为0**；③计数器C_n 的初始值置为 n

进行 n 轮处理

重复 n 轮 **加/减法**、**移位**运算，直到计数器C_n = 0：

1. 将乘数寄存器Y的最低位、辅助位，**2bit** 送入“控制逻辑”进行判断
2. 根据寄存器Y的最低位、辅助位，决定是“ **$+ [x]_{补}$** ” “ **$- [x]_{补}$** ” “ **$+ 0$** ”
3. 将【P, Y, 辅助位】视为整体，**算数右移一位**
4. 计数器C_n减1

Y低	辅	本轮
0	0	$+ 0$
0	1	$+ [x]_{补}$
1	0	$- [x]_{补}$
1	1	$+ 0$

结束

当计数器C_n=0 时，乘法运算结束。用 2n 位【P, Y】暂存乘法运算结果，但最终**仅保留低 n 位【Y】作为最终结果**

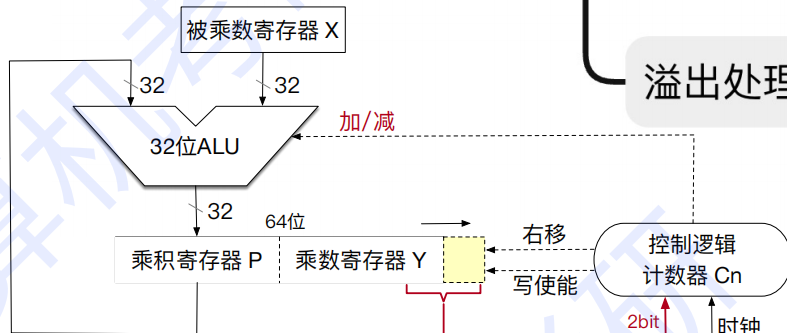
溢出判断

若2n位的**高 n+1 位不完全相同**，说明发生**溢出**，并设置**OF标志位=1**

溢出处理

摆烂：程序员可以选择**忽略乘法溢出**，乘法结果错了也不管

不摆烂：也可以在乘法指令之后执行一条“**溢出自陷指令**”，当OF=1时会触发“**异常处理程序**”来处理溢出



补充一个术语

本节介绍的“补码一位乘法”由 A. D. Booth 提出，因此这种算法也被称为布斯（Booth）乘法。



安德鲁·唐纳德·布斯（Andrew Donald Booth，1918年2月11日—2009年11月29日）是一位英国的工程师、物理学家与计算机科学家。

生平 [编辑]

他曾领导了用于计算机的磁鼓存储器的研发；此外，他还发明了布斯乘法算法。在1943年到1945年间，布斯是英国橡胶生产者研究协会（BRPRA）下辖X射线团队的一名数学物理学家。1945年后，他转职到了伦敦大学的柏贝克学院。

家庭 [编辑]

妻子是计算机科学家凯瑟琳·布斯。